PRODUCT BRIEF · TRACK 5 — HEALTHCARE

ClearNote

Clinical Notes Structurer

From Workflow Pain to a Working AI Product

Turns a clinician's rushed shorthand into a clean four-part note — Symptoms, History, Observations, Next steps — flags critical information that is missing, shows how confident it is, and refuses to add anything the clinician did not write.

Author	Christine Massey
Date	25 October 2026
Stack	Python (Flask) · Claude API (Sonnet 5.5) · SQLite / Replit PostgreSQL · Replit · GitHub
Deliverables	This brief · public Replit prototype · GitHub repo (6 stage commits) · demo video

Contents

1. Problem Statement 2. User Persona 3. JTBD Statement 4. Why AI? 5. Feature List + Value vs Effort 6. Failure Mode Analysis 7. AI Trade-offs 8. Cost & Sustainability 9. Reflection Appendix: System flow

1. Problem Statement

In a busy Indian outpatient clinic a doctor may see 40–60 patients a day, with only a few minutes each. Notes are written in the gaps — in shorthand, in whatever order the thoughts arrived. A real example of what that looks like:

```
54M c/o chest pain radiating to L arm x 2 hrs, sweating ++. h/o HTN, DM on
metformin. O/E BP 186/110, HR 112, SpO2 94%. chest clear. plan ECG stat, trop, refer
cardiology.
```

The note is fast to write, but it fails the moment someone needs to **reuse** it:

- **Continuity of care** — at follow-up, the same or a different clinician has to decode it again.
- **Communication** — referral letters, discharge summaries and handovers need the information in a standard order, so it is rewritten by hand.
- **Gaps stay invisible** — in the example, allergy status is never recorded. Nobody notices until it matters (before a prescription), or until an insurer or TPA sends a query asking for it.
- **Inconsistent quality** — every clinician has their own shorthand, so the clinic cannot see which documentation habits cause problems.

> **Problem in one line:** clinical notes are captured fast but are unstructured, incomplete in ways nobody can see, and costly to reuse — and the fix cannot add work or add risk.

Why now / why me: from eight years in claims and customer-experience operations, I have seen what incomplete documentation costs downstream: claims queried or delayed because a detail was never written down. The cheapest place to catch a gap is the moment the note is written.

2. User Persona

	Primary user
Who	**Dr. Ananya Rao**, 38, family physician. Runs morning OPD at a multi-specialty clinic in Noida and evening private practice. (Fictional persona built from the brief and typical OPD workflows.)
Day	45–60 patients a day, ~4–6 minutes each. Types notes into a free-text box in the clinic software between patients, often finishing them after the queue clears.
Writes like	Heavy shorthand (c/o, h/o, O/E, NKDA), no fixed order, vitals mixed with history.
Goals	Finish notes fast; have them make sense next week; send clean referrals; avoid queries from insurers and other doctors.
Frustrations	Re-reading her own notes at follow-up; rewriting notes into letters; being asked for information she thought she wrote; any tool that adds clicks.
Tech comfort	Uses a smartphone and clinic software daily; no interest in learning a complex tool.
What she must trust	That the tool **never puts words in her mouth**. One invented drug or diagnosis in a note she signs, and she will stop using it.

Secondary users: ward nurses writing handover notes (same pain, shorter notes) and the **clinic admin / quality lead**, who needs to see documentation quality across the team — served by the admin dashboard.

3. JTBD Statement

> **When** I write clinical notes after a consultation, **I want** them structured clearly, **so** they are useful for future care and communication.

Trigger moment: the consultation has just ended; the notes are unstructured and the next patient is waiting.

What 'done well' means for her (the hiring criteria for this product):

Dimension	Success looks like
Functional	Four clear sections in under 15 seconds, from notes exactly as she wrote them.
Safety	Nothing added that she did not write. Every line traceable back to her words.
Completeness	Missing critical items (allergies, vitals, plan, duration, medications) flagged while she can still fix them.
Emotional	Confidence that the structured note is hers and is safe to sign.
Social	Referrals and handovers look professional; fewer questions from colleagues and insurers.

What she 'fires' today: rewriting notes by hand, templates she skips because they are slow, and simply leaving notes unstructured.

4. Why AI? (critical thinking)

The honest answer is: **AI for one step only.** Most of the job is better done with plain rules, and the product is designed that way. For each part of the job I asked: can a rule do this reliably?

Part of the job	Rules or AI?	Why
Validate the form	Rules	Fixed requirements; must be instant and predictable.
Remove names / phones / IDs	Rules	Must happen before any external call. Patterns are known.
Read vitals, check safe ranges	Rules	Numbers and thresholds are exact; AI adds nothing but risk.
Spot red-flag words (e.g. chest pain)	Rules	Safety-critical, must be explainable; keyword list with negation check.
Check for missing critical info	Rules (+ AI extra)	Core checklist is fixed; AI may notice context-specific gaps.
Decide if notes are worth processing	Rules	Stops junk input before it costs money.
Sort free-text shorthand into 4 sections	**AI**	Language is messy, unordered and context-dependent ('chest clear' is an observation; 'h/o asthma' is history). Rules break on every new writing style. This is a language-understanding task.
Expand shorthand into readable sentences	AI (checked)	Needs context; output is verified against the notes.
Verify AI output, score confidence	Rules	AI must not grade its own work.

Where AI is NOT needed and is not used: diagnosis, treatment suggestions, filling in missing values, or deciding urgency. These would add risk without serving the JTBD — the job is *structuring what was*

written, not clinical decision-making.

The test I applied: if the AI step were removed, the product would still validate, protect privacy, check vitals, flag red flags and list gaps — but it could not re-organise the language. That is the exact part where AI earns its place.

5. Feature List + Value vs Effort

Feature	Value	Effort	Decision
Structured input form (patient code, age, sex, visit type, role, notes)	High	Low	MVP
Input validation with clear messages	High	Low	MVP
Logic layer: privacy redaction before the AI	High	Low	MVP
Logic layer: vitals extraction + safe-range check	High	Medium	MVP
Logic layer: red-flag banner (with negation check)	High	Low	MVP
Logic layer: missing-critical-info checklist	High	Low	MVP
Routing: stop non-clinical / too-thin notes before AI	Medium	Low	MVP
AI structuring into 4 sections with evidence quotes	High	Medium	MVP
Grounding check that blocks invented items	High	Medium	MVP — critical
Confidence score with plain-English reasons	High	Low	MVP
Database storage of every submission	High	Low	MVP
Saved-notes list + 'Mark as reviewed'	Medium	Low	MVP
Admin dashboard (quality, safety, cost)	Medium	Medium	MVP
Copy as text (for referral letters)	Medium	Low	MVP
Voice dictation input	High	High	Later
Generate referral / discharge letter from the structured note	High	Medium	Later
Integration with clinic EMR software	High	High	Later
Edit structured note inline before saving	Medium	Medium	Later
User accounts and roles per clinician	Medium	Medium	Later
Hindi / mixed-language notes	Medium	High	Later

Prioritisation logic: everything that protects safety and trust is in the MVP even when it costs effort (grounding check). Features that save more time but add integration risk (voice, EMR) come after the core is proven safe.

6. Failure Mode Analysis

Key failure mode (critical): the AI invents medical details that are not in the input. This is handled with four independent layers, so no single failure lets an invention through:

Layer	What it does
1. Prompt rules	Claude is told it is a formatter, not a doctor; must use only the notes; must return an exact evidence quote for every item; empty sections are correct. No room to improvise.
2. Quote check	Every evidence quote is searched for in the original notes. Not found = item blocked.
3. Number check	Any number in an item (dose, value, duration) must exist in the notes. Otherwise blocked.
4. Word check	If most meaningful words in an item cannot be traced to the notes (e.g. an invented drug or diagnosis attached to a real quote), the item is blocked.
Visible to user	Blocked items are shown struck-through with the reason, lower the confidence score, and are counted on the admin dashboard.

Tested example: a simulated AI reply that added 'Aspirin 300 mg given', 'Smoker for 20 years' and 'Possible myocardial infarction' — none in the notes — had all three blocked, while legitimate expansions such as 'ECG stat' to 'ECG immediately' were kept.

Full failure-mode table

Failure mode	Impact	Likely?	Mitigation in the product
AI invents a symptom, drug, value or diagnosis	Critical	Medium	4-layer grounding check above.
AI puts an item in the wrong section	Medium	Medium	Evidence quote shown under every item so the clinician can spot it; per-section confidence; 'Needs review' status.
Critical gap not noticed	High	Medium	Rule-based checklist runs on every note regardless of AI; AI can add extra gaps; both shown with severity.
Private data sent to an external AI	High	Medium	Form asks for a patient code, not a name; identifiers redacted by rules before the API call; only redacted text is stored.
Red flag missed, or false alarm from 'no chest pain'	High	Low	Rule-based red-flag list with negation check ('no', 'denies', 'nil'); banner shown above everything else.
Vital sign typo (e.g. SpO2 190%)	Medium	Low	Range check marks impossible values 'please re-check'.
Junk / non-clinical text submitted	Low	Medium	Routing stops it before the AI: zero cost, clear message.
AI service down, slow, key invalid, or broken reply	Medium	Low	Timeout + retry, one format retry, friendly error; rule-based results still shown; failure saved and counted.
Over-trust: clinician signs without reading	High	Medium	Disclaimer on every page; confidence reasons; 'Mark as reviewed' tracked; dashboard shows notes awaiting review.
Instructions hidden inside notes (prompt injection)	Medium	Low	Notes are wrapped as data in the prompt; output must still pass the grounding check, so injected content cannot add facts.
Database unavailable	Medium	Low	Result still shown to the clinician; error noted; automatic fallback to a local database file.

Failure mode	Impact	Likely?	Mitigation in the product
Grounding check too strict (blocks a valid paraphrase)	Low	Medium	Accepted trade-off: blocked items stay visible, so nothing is silently lost.

7. AI Trade-offs

Decision	Chosen	Gave up	Why
Model	Claude Sonnet 5.5	Lower cost of Haiku 5.5	Accuracy in following strict rules matters more than a fraction of a cent per note. Model is a setting, so Haiku can be tested later.
Consistency	Strict rules + checks	A model setting for randomness	The current Claude API has no temperature setting, so consistency comes from strict prompt rules and the grounding check, which judges every output the same way.
Output format	JSON with evidence per item	Simple free text	Makes parsing reliable and makes every line checkable. User never sees the JSON.
Safety vs completeness	Strict grounding	Some valid paraphrases	A missing line is visible and fixable; an invented line is dangerous. Blocked items are shown, not hidden.
Rules vs AI	Rules for checks, AI for language	Doing everything with AI (simpler to build)	Rules are free, instant, explainable and testable. AI is used only where language understanding is needed.
Confidence signal	Calculated score	Trusting the AI's own rating	Score is 45% verified-item rate, 25% AI self-rating, 30% section coverage, minus penalties for blocked items, critical gaps and unclear points.
Privacy	Redact before sending	Some context (e.g. names)	Names add no clinical value to structuring and should never leave the clinic.
Speed	One AI call per note (~5–15 s)	Instant results	One call keeps cost and latency low; rule-based results appear even if AI fails.

8. Cost & Sustainability

Pricing used (Claude API, October 2026, per million tokens): **Sonnet 5.5** $2 input / $10 output; **Haiku 5.5** $0.10 input / $0.50 output (for prompts up to 100,000 tokens). Source: Anthropic pricing, cross-checked with published price trackers.

Estimate per note	Sonnet 5.5	Haiku 5.5
Input ~1,200 tokens (rules + context + notes)	$0.0024	$0.00012
Output ~700 tokens (4 sections + evidence + flags)	$0.0070	$0.00035
Cost per note	**~$0.009 (under 1 rupee)**	**~$0.0005**
One doctor, 50 notes/day, 26 days (1,300 notes/month)	~$12 / month	~$0.60 / month
Clinic of 10 doctors (13,000 notes/month)	~$120 / month	~$6 / month

What keeps it sustainable

- **Logic layer first:** non-clinical or too-thin notes are stopped before the AI — they cost nothing.

- **One call per note**, capped output (2,000 tokens), notes capped at 6,000 characters.
- **Cost tracked per note** (tokens, seconds, dollars) and totalled on the admin dashboard, so spend is visible from day one.
- **Model is a setting:** if testing shows Haiku 5.5 is accurate enough, cost drops ~95%.
- **Value comparison:** if structuring saves a doctor even 1 minute per note, 50 notes saves ~50 minutes a day for roughly $0.47 of AI cost.

Other costs: hosting on Replit and a small PostgreSQL database; GitHub is free. **Not in scope for the prototype** but required before real use: data-protection compliance (India's DPDP Act), clinical safety review, and a data-processing agreement with the AI provider.

9. Reflection

What I set out to prove. That a useful AI product starts from a real moment of pain — the minute after a consultation — not from what a model can do. Writing the JTBD first changed the design: the job is *structuring what was written*, so anything that adds information (diagnosis, suggestions) was ruled out before a single prompt was written.

What surprised me. How little of the product is AI. Only one of the ten pipeline steps calls the model; the rest are plain code and rules. The logic layer does privacy, vitals, red flags, gaps and routing — and it keeps working even when the AI fails. The AI's job is narrow, which makes it easier to check.

The hardest problem. Preventing invented details. A prompt rule alone is a promise, not a guarantee. Asking for an evidence quote made every line checkable, but testing showed a gap: the AI could attach a real quote to an invented claim. Adding the number and word checks closed it. Lesson: design the check, then try to break it.

What my background added. In claims work, missing documentation is the most common reason for a query or delay. That is why missing-information flags sit beside the note, with severity — the cheapest time to fix a gap is while the clinician still remembers.

What I would do next. Test with real (anonymised) notes from clinicians and measure how often the grounding check blocks valid lines; try Haiku 5.5 to cut cost; add inline editing and letter generation; and complete a proper privacy and clinical safety review before any real use.

Appendix — System Flow (as built)

#	Step	Built in	AI?
1	Structured input form	templates/index.html	No
2	Input validation	validation.py	No
3	Logic layer (redaction, vitals, red flags, sections, gaps, routing)	logic.py	No
4	Prompt construction (notes + rule findings + strict rules)	ai.py: build_prompt	-
5	LLM API call (Claude, timeout, retry)	ai.py: call_claude	Yes
6	Output parsing (JSON to clean data, format retry)	ai.py: parse_output	No
7	Confidence / flag check (grounding, blocking, scoring)	ai.py: ground_and_score	No
8	Database storage (all submissions)	db.py	No
9	Structured UI output (cards, evidence, flags)	templates/result.html	No
10	Admin dashboard (quality, safety, cost)	/admin	No

GitHub commit plan (one per stage)

#	Commit	Adds
1	Stage 1: Product brief	README, this brief (docs/Product_Brief.pdf)
2	Stage 2: Input form	Flask app, structured form, validation
3	Stage 3: Logic layer	Rule-based checks before AI, results page
4	Stage 4: AI integration	Claude call, parsing, grounding, confidence
5	Stage 5: Database	Storage, saved-notes list, review status
6	Stage 6: Admin dashboard	Password-protected dashboard